🐍 Python Programming for Beginners

Day 2: Variables and Data Types

📘 Course

Python Programming for Beginners (20-Day Course)

📚 Topics Covered

- Variables
- Rules for Naming Variables
- Variable Naming Conventions
- Multiple Variable Assignment
- Constants
- Data Types
- Numeric Data Types (`int` , `float` , `complex`)
- Strings (`str`)
- Boolean (`bool`)
- Collection Data Types
- The `type()` Function
- Type Conversion
- User Input (`input()`)
- Common Errors
- Best Practices
- Practice Programs
- Summary

Learning Objectives

By the end of this lesson, you will be able to:

- Understand the concept of variables in Python.
- Create and assign values to variables.
- Follow the rules for naming variables.
- Use meaningful variable names following Python conventions.

- Identify different Python data types.
- Use the `type()` function to check the data type of a variable.
- Perform basic type conversion using `int()`, `float()`, `str()`, and `bool()`.
- Accept user input using the `input()` function.
- Write simple Python programs using variables and data types.
- Identify and fix common errors related to variables and data types.
- Follow best practices while writing Python programs.

What is a Variable?

A **variable** is a named memory location used to store data in a Python program. It acts like a container that holds a value, which can be changed during program execution.

Variables make it easy to store, access, and manipulate data without rewriting the values multiple times.

Real-Life Example

Think of a variable as a **labeled box**.

- The **label** is the variable name.
- The **item inside the box** is the value.

For example:

- Box Label → `name`
- Value → `"Shivam"`

Whenever you use `name`, Python retrieves the value `"Shivam"`.

Syntax

```
variable_name = value
```

In [ ]:
```
## Example 1

name = "Alex"
age = 19
city = "London"
```

In [ ]:
```
## Printing Variables
name = "Alex"
age = 19
```

```
print(name)
print(age)
```

Why Do We Use Variables?

- Store data for later use.
- Reuse values without rewriting them.
- Make programs easier to read and maintain.
- Allow values to change during program execution.

Key Points

- A variable is a named container for storing data.
- Variables are created using the assignment operator (`=`).
- Variable values can be changed during program execution.

Variable Declaration

Variable declaration is the process of creating a variable and assigning a value to it. In Python, you do not need to specify the data type. Python automatically determines the data type based on the assigned value.

Syntax

```
variable_name = value
```

- `variable_name` → Name of the variable
- `=` → Assignment operator
- `value` → Data stored in the variable

Example 1

```
name = "Shivam"
age = 19
city = "Patna"
```

Example 2

```
price = 99.99
is_student = True
```

Printing Variables

```
name = "Shivam"
age = 19

print(name)
print(age)
```

Output

```
Shivam
19
```

Changing a Variable

A variable's value can be updated at any time.

In [ ]:
```
age = 19
print(age)

age = 20
print(age)
```

Key Points

- Variables are created using the assignment (`=`) operator.
- Python automatically assigns the data type.
- A variable can store different types of values.
- The value of a variable can be changed during program execution.

Rules for Naming Variables

A variable name must follow Python's naming rules to avoid errors and improve code readability.

Rules

1. Must Start with a Letter or Underscore (_)

✅ Valid

```
name = "Aman"
_age = 19
```

❌ Invalid

```
1name = "Aman"
```

---

2. Can Contain Letters, Numbers, and Underscores

✅ Valid

```
student1 = "Rahul"
student_name = "Aman"
```

❌ Invalid

```
student-name = "Rahul"
student@name = "Shivam"
```

---

3. Cannot Use Spaces

✅ Valid

```
student_name = "Shivam"
```

❌ Invalid

```
student name = "Shivam"
```

---

4. Variable Names are Case-Sensitive

```
name = "Shivam"
Name = "Rahul"
```

`name` and `Name` are two different variables.

---

5. Cannot Use Python Keywords

❌ Invalid

```
if = 10
class = "Python"
```

Summary Table

Rule	Valid Example	Invalid Example
Starts with a letter or `_`	`name`	`1name`
Letters, numbers, `_` only	`student_1`	`student-name`
No spaces	`student_name`	`student name`
Case-sensitive	`age`, `Age`	—
No Python keywords	`marks`	`if`, `class`

Key Points

- Start variable names with a letter or underscore (`_`).
- Use only letters, numbers, and underscores.
- Do not use spaces or special characters.
- Variable names are case-sensitive.
- Avoid using Python keywords as variable names.

Naming Conventions

Naming conventions are recommended styles for naming variables. They make code easier to read, understand, and maintain.

---

1. snake_case (Recommended in Python)

Words are written in lowercase and separated by underscores (`_`).

Example:

```
student_name = "Aman"
total_marks = 95
```

✅ This is the recommended naming style in Python (PEP 8).

---

2. camelCase

The first word starts with a lowercase letter, and each following word starts with a capital letter.

Example:

```
studentName = "Alex"
totalMarks = 95
```

Commonly used in JavaScript and Java.

---

3. PascalCase

Each word starts with a capital letter.

Example:

```
StudentName = "rahul"
TotalMarks = 95
```

Commonly used for **class names** in Python.

Comparison Table

Convention	Example	Used For
snake_case	`student_name`	Variables and functions (Recommended)
camelCase	`studentName`	Java, JavaScript
PascalCase	`StudentName`	Class names

Best Practices

- Use meaningful and descriptive names.
- Use **snake_case** for variables and functions.
- Use **PascalCase** for class names.
- Avoid short names like `a`, `b`, or `x` unless they are used temporarily.

Key Points

- **snake_case** is the standard naming convention in Python.
- Use names that clearly describe the purpose of the variable.
- Following naming conventions improves code readability and maintainability.

Multiple Variable Assignment

Python allows you to assign values to multiple variables in a single line. This makes the code shorter and more readable.

Syntax

```
variable1, variable2, variable3 = value1, value2, value3
```

Example 1: Assign Different Values

In [ ]:
```
name, age, city = "Alex", 19, "London"

print(name)
print(age)
print(city)
```

Output

```
Alex
19
London
```

Example 2: Assign the Same Value

You can assign the same value to multiple variables.

In [ ]:
```
x = y = z = 100

print(x)
print(y)
print(z)
```

Output

```
100
100
100
```

Example 3: Swap Two Variables

Python allows you to swap two variables without using a temporary variable.

In [ ]:
```
a = 10
b = 20
a, b = b, a
```

```
print(a)
print(b)
```

Output

```
20
10
```

Advantages

- Reduces the number of lines of code.
- Improves code readability.
- Makes swapping variables simple.
- Saves time when assigning multiple values.

Key Points

- Assign multiple values using commas (`,`).
- The number of variables and values must be the same.
- You can assign the same value to multiple variables.
- Python supports variable swapping without a temporary variable.

Introduction to Data Types

A **data type** specifies the type of data that a variable can store. It helps Python understand how to store, process, and manipulate different kinds of values.

In Python, you do not need to declare the data type explicitly. Python automatically determines the data type based on the value assigned to the variable. This feature is called **dynamic typing**.

Why Do We Need Data Types?

- Store different kinds of data.
- Perform correct operations on data.
- Prevent invalid operations and errors.
- Improve code readability and efficiency.

Example

```
age = 19
price = 99.99
name = "Shivam"
is_student = True
```

Here:

- `19` is an **Integer (`int`)**
- `99.99` is a **Float (`float`)**
- `"Shivam"` is a **String (`str`)**
- `True` is a **Boolean (`bool`)**

Categories of Python Data Types

Category	Data Types
Numeric	`int` , `float` , `complex`
Text	`str`
Boolean	`bool`
Sequence	`list` , `tuple` , `range`
Set	`set` , `frozenset`
Mapping	`dict`
Binary	`bytes` , `bytearray` , `memoryview`
None Type	`NoneType`

Key Points

- Every value in Python has a data type.
- Python automatically identifies the data type.
- Different data types are used to store different kinds of information.
- Choosing the correct data type makes programs more efficient and easier to understand.

Numeric Data Types

Numeric data types are used to store numerical values. Python provides three built-in numeric data types:

- **int** (Integer)

- **float** (Floating-point)
- **complex** (Complex Number)

---

1. Integer (`int`)

An **integer** is a whole number without a decimal point. It can be positive, negative, or zero.

Example

```
age = 19
temperature = -5
count = 100

print(age)
print(temperature)
print(count)
```

Output

```
19
-5
100
```

---

2. Floating-Point (`float`)

A **float** is a number that contains a decimal point.

Example

```
price = 99.99
height = 5.8

print(price)
print(height)
```

Output

```
99.99
5.8
```

---

3. Complex (`complex`)

A **complex** number contains a real part and an imaginary part. The imaginary part is represented using `j`.

Example

```
num = 3 + 4j

print(num)
```

Output

```
(3+4j)
```

Comparison Table

Data Type	Description	Example
`int`	Whole numbers	`10` , `-5` , `0`
`float`	Decimal numbers	`3.14` , `99.5`
`complex`	Real and imaginary numbers	`2+3j` , `5-1j`

Check Numeric Data Types

```
print(type(10))
print(type(3.14))
print(type(2 + 3j))
```

Output

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

Key Points

- `int` stores whole numbers.
- `float` stores decimal numbers.
- `complex` stores numbers with real and imaginary parts.
- Use the `type()` function to identify the data type of a value.

In [ ]:
```
print(type(10))
print(type(3.14))
print(type(2 + 3j))
```

String (`str`)

A **string** is a sequence of characters enclosed in **single (`' '`)** or **double (`" "`)** quotation marks. Strings are used to store text such as names, addresses, and messages.

Creating a String

Using Double Quotes

```
name = "Aman"
```

Using Single Quotes

```
city = 'Nagpur'
```

Both are valid in Python.

Example

```
name = "Aman"
college = "Delhi University"

print(name)
print(college)
```

Output

```
Aman
Delhi University
```

Multi-Line String

You can create a multi-line string using triple quotes.

```
message = """
Welcome to
Python Programming
"""
print(message)
```

Output

```
Welcome to
Python Programming
```

Check the Data Type

```
text = "Hello"

print(type(text))
```

Output

```
<class 'str'>
```

Characteristics of Strings

- Stores text or characters.
- Enclosed in single or double quotes.
- Can contain letters, numbers, and symbols.
- Strings are immutable (cannot be changed after creation).

Examples

Value	Data Type
`"Python"`	`str`
`'Hello'`	`str`
`"123"`	`str`
`"Welcome to Python"`	`str`

Key Points

- A string is a collection of characters.
- Strings are enclosed in quotation marks.
- Use the `type()` function to check if a value is a string.
- Strings are one of the most commonly used data types in Python.

Boolean (`bool`)

A **Boolean** data type represents one of two possible values:

- `True`
- `False`

Boolean values are commonly used in decision-making, comparisons, and conditional statements.

Creating Boolean Variables

```
is_student = True
is_logged_in = False
```

Example

```
is_student = True
is_logged_in = False

print(is_student)
print(is_logged_in)
```

Output

```
True
False
```

Check the Data Type

```
status = True

print(type(status))
```

Output

```
<class 'bool'>
```

Boolean from Comparison

Comparison operators return Boolean values.

```
print(10 > 5)
print(10 < 5)
print(10 == 5)
```

Output

```
True
False
False
```

Characteristics of Boolean

- Has only two values: `True` and `False`.

- Used in comparisons and decision-making.
- `True` and `False` are case-sensitive.

Examples

Value	Data Type
`True`	`bool`
`False`	`bool`
`10 > 5`	`bool`
`3 == 7`	`bool`

Key Points

- A Boolean stores either `True` or `False`.
- Use the `type()` function to check the Boolean data type.
- Boolean values are widely used with conditions, loops, and comparison operators.

Collection Data Types

Collection data types are used to store **multiple values** in a single variable. Python provides several built-in collection data types.

The four main collection data types are:

- **List (`list`)**
- **Tuple (`tuple`)**
- **Set (`set`)**
- **Dictionary (`dict`)**

1.List (`list`)

A **list** is a collection used to store multiple items in a single variable. Lists are one of the most commonly used data types in Python.

Lists are:

- **Ordered** – Items have a defined order.

- **Changeable (Mutable)** – Items can be modified after creation.
- **Allow Duplicate Values** – The same value can appear more than once.

Lists are created using **square brackets (`[]`)**.

Example

```
fruits = ["Apple", "Banana", "Mango"]

print(fruits)
```

Output

```
['Apple', 'Banana', 'Mango']
```

Check the Data Type

```
fruits = ["Apple", "Banana", "Mango"]

print(type(fruits))
```

Output

```
<class 'list'>
```

Key Points

- Stores multiple items in one variable.
- Created using square brackets `[]`.
- Ordered and changeable.
- Allows duplicate values.
- More details will be covered in a dedicated lesson on Lists.

2.Tuple (`tuple`)

A **tuple** is a collection used to store multiple items in a single variable. Tuples are similar to lists, but they **cannot be changed** after they are created.

Tuples are:

- **Ordered** – Items have a defined order.
- **Unchangeable (Immutable)** – Items cannot be modified after creation.
- **Allow Duplicate Values** – The same value can appear more than once.

Tuples are created using **parentheses (`()`)**.

Example

```
fruits = ("Apple", "Banana", "Mango")

print(fruits)
```

Output

```
('Apple', 'Banana', 'Mango')
```

Check the Data Type

```
fruits = ("Apple", "Banana", "Mango")

print(type(fruits))
```

Output

```
<class 'tuple'>
```

Key Points

- Stores multiple items in one variable.
- Created using parentheses `()`.
- Ordered and unchangeable.
- Allows duplicate values.
- More details will be covered in a dedicated lesson on Tuples.

3.Set (`set`)

A **set** is a collection used to store multiple unique items in a single variable.

Sets are:

- **Unordered** – Items do not have a fixed order.
- **Unchangeable*** – Individual items cannot be changed, but you can add or remove items.
- **Do Not Allow Duplicate Values** – Duplicate items are automatically removed.

Sets are created using **curly braces (`{}`)**.

Example

```
fruits = {"Apple", "Banana", "Mango"}

print(fruits)
```

Output

```
{'Apple', 'Banana', 'Mango'}
```

> **Note:** The order of items in a set may be different each time you run the program.

Duplicate Values

```
fruits = {"Apple", "Banana", "Apple"}

print(fruits)
```

Output

```
{'Apple', 'Banana'}
```

Check the Data Type

```
fruits = {"Apple", "Banana", "Mango"}

print(type(fruits))
```

Output

```
<class 'set'>
```

Key Points

- Stores multiple unique items in one variable.
- Created using curly braces `{}`.
- Unordered collection.
- Duplicate values are not allowed.
- More details will be covered in a dedicated lesson on Sets.

4.Dictionary (`dict`)

A **dictionary** is a collection used to store data in **key-value pairs**. Each key is unique and is used to access its corresponding value.

Dictionaries are:

- **Ordered** – Items have a defined order.
- **Changeable (Mutable)** – Items can be added, removed, or modified.
- **Do Not Allow Duplicate Keys** – Each key must be unique.

Dictionaries are created using **curly braces (`{}`)**.

Example

```
student = {
    "name": "Shivam",
    "age": 19,
    "course": "CSE"
}

print(student)
```

Output

```
{'name': 'Shivam', 'age': 19, 'course': 'CSE'}
```

Check the Data Type

```
student = {
    "name": "Shivam",
    "age": 19
}

print(type(student))
```

Output

```
<class 'dict'>
```

Key Points

- Stores data as **key-value pairs**.
- Created using curly braces `{}`.
- Ordered and changeable.
- Keys must be unique.
- More details will be covered in a dedicated lesson on Dictionaries.

Type Checking

Python provides the built-in `type()` function to check the data type of a variable or value.

Syntax

```
type(object)
```

Example 1

In [ ]:
```
## Example 1

age = 19
price = 99.99
name = "Shivam"
is_student = True

print(type(age))
print(type(price))
print(type(name))
print(type(is_student))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Example 2

In [ ]:
```
## Example 2

fruits = ["Apple", "Banana"]
student = {"name": "Shivam"}

print(type(fruits))
print(type(student))
```

Output

```
<class 'list'>
<class 'dict'>
```

Common Data Types

Value	Data Type
`10`	`int`
`3.14`	`float`
`"Python"`	`str`
`True`	`bool`
`[1, 2, 3]`	`list`
`(1, 2, 3)`	`tuple`
`{1, 2, 3}`	`set`
`{"a": 1}`	`dict`

Key Points

- Use the `type()` function to check the data type of a variable or value.
- It returns the class (type) of the object.
- Type checking helps verify that a variable contains the expected kind of data.

Type Conversion

Type conversion is the process of converting a value from one data type to another. Python provides built-in functions to perform type conversion.

Common Type Conversion Functions

Function	Converts To
`int()`	Integer (`int`)
`float()`	Floating-point (`float`)
`str()`	String (`str`)
`bool()`	Boolean (`bool`)

Convert to Integer

In [ ]:
```
x = "100"
```

```
print(int(x))
```

Output

```
100
```

Convert to Float

In [ ]:
```
x = 10

print(float(x))
```

Output

```
10.0
```

Convert to String

In [ ]:
```
x = 25

print(str(x))
```

Output

```
25
```

Convert to Boolean

In [ ]:
```
print(bool(1))
print(bool(0))
```

Output

```
True
False
```

Check the Data Type

In [ ]:
```
x = "100"
print(type(x))

x = int(x)
print(type(x))
```

Output

```
<class 'str'>
<class 'int'>
```

Key Points

- Type conversion changes one data type into another.
- Use `int()`, `float()`, `str()`, and `bool()` for conversion.
- Convert data only when it is compatible with the target data type.
- Use `type()` to verify the converted data type.

Input Function

The **`input()`** function is used to accept input from the user during program execution. It allows the user to enter data using the keyboard.

By default, the `input()` function returns the entered value as a **string (`str`)**.

Syntax

```
input("Prompt Message")
```

Example 1

In [ ]:
```
name = input("Enter your name: ")
print(name)
```

Input

```
Aman
```

Output

```
Aman
```

Example 2

In [ ]:
```
city = input("Enter your city: ")
print("City:", city)
```

Input

```
Nagpur
```

Output

```
City: Nagpur
```

Input Returns a String

In [ ]:
```
age = input("Enter your age: ")
print(type(age))
```

Input

```
19
```

Output

```
<class 'str'>
```

Converting Input to Integer

In [ ]:
```
age = int(input("Enter your age: "))
print(type(age))
```

Input

```
19
```

Output

```
<class 'int'>
```

Key Points

- Use `input()` to accept input from the user.
- `input()` always returns a **string**.
- Use `int()`, `float()`, or other conversion functions when a different data type is required.
- The prompt message helps users understand what they need to enter.

Common Errors

Beginners often make small mistakes while working with variables, data types, and user input. Understanding these errors helps you write better Python programs.

---

1. NameError

This error occurs when a variable is used before it is created or when the variable name is misspelled.

Example

```
print(name)
```

Output

```
NameError: name 'name' is not defined
```

---

2. SyntaxError

This error occurs when Python code does not follow the correct syntax.

Example

```
print("Hello"
```

Output

```
SyntaxError: '(' was never closed
```

---

3. TypeError

This error occurs when an operation is performed on incompatible data types.

Example

```
age = "19"

print(age + 5)
```

Output

```
TypeError: can only concatenate str (not "int") to str
```

---

4. ValueError

This error occurs when a value cannot be converted to the required data type.

Example

```
age = int("abc")
```

Output

```
ValueError: invalid literal for int()
```

---

5. IndentationError

This error occurs when the indentation is incorrect.

Example

```
if True:
print("Hello")
```

Output

```
IndentationError: expected an indented block
```

Tips to Avoid Errors

- Use meaningful variable names.
- Check spelling carefully.
- Match opening and closing brackets.
- Convert data to the correct type before using it.
- Follow proper indentation.
- Read error messages carefully to identify the problem.

Practice Programs (10)

Solve the following programs on your own before checking the solution.

Program 1

Write a Python program to store your name in a variable and print it.

Program 2

Write a Python program to store a person's name, age, and city in separate variables and display them.

Program 3

Write a Python program to store an integer, a float, a string, and a boolean value. Print all the values.

Program 4

Write a Python program to check the data type of the following values using the `type()` function:

- `100`
- `25.75`
- `"Python"`
- `True`

Program 5

Write a Python program to convert the string `"250"` into an integer and print its data type.

Program 6

Write a Python program to convert the integer `50` into a float and display the result.

Program 7

Write a Python program to take a user's name as input and display a welcome message.

Example Output

```
Welcome, Alex!
```

Program 8

Write a Python program to take a user's age as input and display it.

Program 9

Write a Python program to take a user's name and city as input and display them.

Example Output

```
Name: Alex
City: London
```

Program 10

Write a Python program to take a user's name, age, and favorite programming language as input and display the information in the following format.

Example Output

```
User Details
------------
Name      : Alex
Age       : 20
Language  : Python
```

Summary

In this lesson, you learned the fundamentals of **variables** and **data types** in Python.

In this lesson, you learned:

- What a variable is and how to declare it.
- Rules and naming conventions for variables.
- How to assign values to single and multiple variables.
- The concept of constants in Python.
- Different Python data types:
  - `int`
  - `float`
  - `complex`
  - `str`
  - `bool`
  - `list`
  - `tuple`
  - `set`
  - `dict`
- How to check the data type using the `type()` function.
- How to convert one data type to another using:
  - `int()`
  - `float()`
  - `str()`
  - `bool()`
- How to accept user input using the `input()` function.

- Common beginner errors and how to avoid them.
- Simple programs using variables, data types, and user input.

Key Takeaways

- Variables store data in memory.
- Python automatically determines the data type of a variable.
- Choose meaningful variable names and follow Python naming conventions.
- Use `type()` to identify the data type of a value.
- Use type conversion functions when changing data types.
- The `input()` function always returns a string unless converted.
- Practice is the best way to strengthen your understanding of variables and data types.

What's Next?

In **Day 3**, you will learn about **Python Operators**, including:

- Arithmetic Operators
- Assignment Operators
- Comparison Operators
- Logical Operators
- Identity Operators
- Membership Operators
- Bitwise Operators

Happy Coding! 🚀